

# Laboratory 1 — Environment Setup: Anaconda, Jupyter Notebook and Git

**Course:** Introduction to Artificial Intelligence (AI-101L) · **Instructor:** Dr. Muhammad Awais

**Duration:** 3 hours · **Block:** Phase A — Environment and Data · **Mapped CLO:** CLO-1

---

## Learning Objectives

On completion of this laboratory you will be able to:

1. Explain what Anaconda and Jupyter Notebook are, and why they are the standard platform for a Python-based data science workflow.
2. Install Anaconda and create an isolated `conda` environment for the whole course.
3. Launch Jupyter Notebook, write and execute Python code in cells, and export the finished notebook.
4. Load a dataset with `pandas` and inspect it using `head()`, `info()` and `describe()`.
5. Explain version control, and use Git and GitHub to commit, branch, merge and publish your laboratory portfolio.

## How to use this notebook

Run every cell **in order**, top to bottom, with `Shift + Enter`. Cells that begin with `%%bash` or a `#` comment block describe commands you run in the **Anaconda Prompt / terminal**, not inside the notebook — read them, run them in a terminal, and paste the output into the answer cell that follows.

**Instructor note.** Sections 1–4 are demonstrated live. Section 5 (Git) is worked through by each student on their own machine; the deliverable is a GitHub repository URL.

---

## 1. Background

### 1.1 What is Anaconda?

Anaconda is an open-source **distribution** of the Python and R programming languages, tailored for scientific computing, data analysis and machine learning. It simplifies package

management and deployment, and ships with a large collection of pre-installed libraries.

Its four principal advantages are:

| Advantage                                                                                  | Why it matters to you                                  |
|--------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Bundles the essential data-science packages ( NumPy , pandas , matplotlib , scikit-learn ) | No manual dependency resolution on day one             |
| Includes integrated tools: Jupyter Notebook, Spyder, Anaconda Navigator                    | One install gives you the whole workbench              |
| Provides conda , which manages packages <b>and</b> isolated environments                   | Two projects with conflicting requirements can coexist |
| Cross-platform                                                                             | Windows, macOS and Linux behave identically            |

## 1.2 What is Jupyter Notebook?

Jupyter Notebook is a **browser-based interactive development environment** in which Python code is written and executed in a sequence of *cells*. A notebook interleaves executable code, its output, and rich documentation (Markdown, equations, plots) in a single file.

This makes it especially suitable for analysis, reporting and teaching, because:

- it supports step-by-step coding and analysis in segmented cells;
- it renders data visualisations inline, next to the code that produced them;
- it encourages reproducibility and transparency in analytical work;
- it is the de facto standard in both academia and industry.

## 1.3 The tool chain you are building today

```

Operating System  ->  Anaconda  ->  conda env 'ai-lab'  ->  Jupyter
Notebook  ->  Git + GitHub
    Windows /          Python 3.11    isolated packages    cells,
    Markdown,      versioned
    macOS / Linux  + conda            for all 16 labs      plots
portfolio

```

Anaconda manages the software ecosystem; Jupyter provides the interface for exploratory programming; Git records every step so nothing is ever lost.

## 2. Procedure

## Task 1.1 — Install Anaconda

Download the installer for your operating system from <https://www.anaconda.com/download> and complete the installation with the default options.

On Windows, accept the default of *not* adding Anaconda to `PATH` and use the **Anaconda Prompt** instead. Verify the installation by opening Anaconda Navigator and confirming that Jupyter Notebook is listed.

Then run the commands in the next cell **in the Anaconda Prompt** (not in this notebook).

```
In [1]: # =====
# RUN THESE IN THE ANACONDA PROMPT (Windows) OR A TERMINAL (macOS / Linux).
# They are shown here as a reference; they are commented out so that running
# this cell by accident cannot alter your machine.
# =====

# conda --version
# python --version

# Create ONE isolated environment that will be used for all sixteen labs
# conda create -n ai-lab python=3.11 -y
# conda activate ai-lab

# Register the environment as a Jupyter kernel so notebooks can select it
# pip install ipykernel
# python -m ipykernel install --user --name ai-lab --display-name "Python (ai-Lab)"

print("Read the commands above, run them in the Anaconda Prompt, then continue.")
```

Read the commands above, run them in the Anaconda Prompt, then continue.

### Why an environment, and not the base installation?

Installing every package into `base` guarantees that a version bump required by Laboratory 8 will silently break Laboratory 4. An environment is cheap to create and cheap to discard; `base` is neither.

```
In [2]: # Verify that THIS notebook is running inside the environment you just created.
# If the executable path does not contain "ai-lab", change the kernel:
#     Kernel -> Change kernel -> Python (ai-Lab)

import sys
import platform

print("Python version   :", sys.version.split()[0])
print("Interpreter path :", sys.executable)
print("Platform         :", platform.platform())
```

```
print()
print("Running in ai-lab:", "ai-lab" in sys.executable)
```

```
Python version      : 3.13.5
Interpreter path    : C:\Program Files\Python313\python.exe
Platform           : Windows-10-10.0.19045-SP0
```

```
Running in ai-lab: False
```

## Task 1.2 — Launch Jupyter and create your first notebook

Launch Jupyter Notebook either from Anaconda Navigator or from the prompt with:

```
jupyter notebook
```

Create a new notebook, select the **Python (ai-lab)** kernel, and rename it

`lab01_setup.ipynb`. The three cells below are the first three cells every student writes.

```
In [3]: # Cell 1 -- the classic first statement
print("Hello, Introduction to Artificial Intelligence")
```

```
Hello, Introduction to Artificial Intelligence
```

```
In [4]: # Cell 2 -- installing a library from inside the notebook.
# The leading ! sends the line to the system shell rather than to Python.
# (Already satisfied if you installed pandas with the environment.)

!pip install pandas
```

```
Requirement already satisfied: pandas in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (3.0.3)
Requirement already satisfied: numpy>=1.26.0 in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (from pandas) (2.4.6)
Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (from pandas) (2026.2)
Requirement already satisfied: six>=1.5 in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

```
[notice] A new release of pip is available: 26.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
In [5]: # Cell 3 -- confirming the installation
import pandas as pd

print("pandas version:", pd.__version__)
```

```
pandas version: 3.0.3
```

## Task 1.3 — Load and inspect a dataset

Using `pandas`, load a CSV **directly from a URL** and perform the three standard first inspections. Every dataset used in this course is inspected in exactly this way before anything else is done to it.

```
In [6]: import pandas as pd

url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv"
df = pd.read_csv(url)

print("Shape (rows, columns):", df.shape)
df.head()
```

Shape (rows, columns): (891, 15)

```
Out[6]:
```

|   | survived | pclass | sex    | age  | sibsp | parch | fare    | embarked | class | who   | adult_m |
|---|----------|--------|--------|------|-------|-------|---------|----------|-------|-------|---------|
| 0 | 0        | 3      | male   | 22.0 | 1     | 0     | 7.2500  | S        | Third | man   | T       |
| 1 | 1        | 1      | female | 38.0 | 1     | 0     | 71.2833 | C        | First | woman | Fa      |
| 2 | 1        | 3      | female | 26.0 | 0     | 0     | 7.9250  | S        | Third | woman | Fa      |
| 3 | 1        | 1      | female | 35.0 | 1     | 0     | 53.1000 | S        | First | woman | Fa      |
| 4 | 0        | 3      | male   | 35.0 | 0     | 0     | 8.0500  | S        | Third | man   | T       |

## The three standard inspections

| Method                     | What it tells you, and why it matters                                                                                                                          |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>df.head()</code>     | Displays the first five rows. Confirms the file parsed correctly, the header row was recognised, and the delimiter was right.                                  |
| <code>df.info()</code>     | Lists column names, data types and the number of non-null entries. <b>The non-null count is the fastest way to detect missing data.</b>                        |
| <code>df.describe()</code> | Count, mean, standard deviation, min, max and the 25th / 50th (median) / 75th percentiles for numeric columns — central tendency, spread and skew at a glance. |
| <code>df.shape</code>      | The pair (rows, columns). Records the size of the problem before any filtering.                                                                                |

```
In [7]: # 1. STRUCTURE -- the first five rows: are the columns what we expected?
df.head()
```

Out[7]:

|   | survived | pclass | sex    | age  | sibsp | parch | fare    | embarked | class | who   | adult_m |
|---|----------|--------|--------|------|-------|-------|---------|----------|-------|-------|---------|
| 0 | 0        | 3      | male   | 22.0 | 1     | 0     | 7.2500  | S        | Third | man   | T       |
| 1 | 1        | 1      | female | 38.0 | 1     | 0     | 71.2833 | C        | First | woman | Fa      |
| 2 | 1        | 3      | female | 26.0 | 0     | 0     | 7.9250  | S        | Third | woman | Fa      |
| 3 | 1        | 1      | female | 35.0 | 1     | 0     | 53.1000 | S        | First | woman | Fa      |
| 4 | 0        | 3      | male   | 35.0 | 0     | 0     | 8.0500  | S        | Third | man   | T       |



In [8]: *# 2. COMPOSITION -- column names, dtypes and non-null counts.*  
*# This is where missing values are discovered.*  
 df.info()

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  -
0   survived        891 non-null    int64
1   pclass          891 non-null    int64
2   sex             891 non-null    str
3   age             714 non-null    float64
4   sibsp           891 non-null    int64
5   parch           891 non-null    int64
6   fare            891 non-null    float64
7   embarked        889 non-null    str
8   class           891 non-null    str
9   who             891 non-null    str
10  adult_male      891 non-null    bool
11  deck            203 non-null    str
12  embark_town     889 non-null    str
13  alive           891 non-null    str
14  alone           891 non-null    bool
dtypes: bool(2), float64(2), int64(4), str(7)
memory usage: 116.9 KB
```

In [9]: *# 3. DISTRIBUTION -- count, mean, std, min, max and the 25/50/75 percentiles*  
*# for every numeric column. This reveals scale, spread and outliers.*  
 df.describe()

Out[9]:

|              | survived   | pclass     | age        | sibsp      | parch      | fare       |
|--------------|------------|------------|------------|------------|------------|------------|
| <b>count</b> | 891.000000 | 891.000000 | 714.000000 | 891.000000 | 891.000000 | 891.000000 |
| <b>mean</b>  | 0.383838   | 2.308642   | 29.699118  | 0.523008   | 0.381594   | 32.204208  |
| <b>std</b>   | 0.486592   | 0.836071   | 14.526497  | 1.102743   | 0.806057   | 49.693429  |
| <b>min</b>   | 0.000000   | 1.000000   | 0.420000   | 0.000000   | 0.000000   | 0.000000   |
| <b>25%</b>   | 0.000000   | 2.000000   | 20.125000  | 0.000000   | 0.000000   | 7.910400   |
| <b>50%</b>   | 0.000000   | 3.000000   | 28.000000  | 0.000000   | 0.000000   | 14.454200  |
| <b>75%</b>   | 1.000000   | 3.000000   | 38.000000  | 1.000000   | 0.000000   | 31.000000  |
| <b>max</b>   | 1.000000   | 3.000000   | 80.000000  | 8.000000   | 6.000000   | 512.329200 |

```
In [10]: # Missing values, stated explicitly rather than inferred from info().
missing = df.isnull().sum()
missing = missing[missing > 0].sort_values(ascending=False)

print("Columns containing missing values:\n")
print(missing)
print()
print("As a percentage of all rows:\n")
print((missing / len(df) * 100).round(1))
```

Columns containing missing values:

```
deck          688
age           177
embarked       2
embark_town    2
dtype: int64
```

As a percentage of all rows:

```
deck          77.2
age           19.9
embarked       0.2
embark_town    0.2
dtype: float64
```

## Task 1.4 — Save and export the notebook

From the menu choose **File** → **Save and Export Notebook As** and export the notebook to `.html` and to `.pdf` in addition to the native `.ipynb`.

- The `.ipynb` file is what you **commit to Git**.
- The exported copy is what you **submit for assessment**.

## 3. Git and Version Control

### 3.1 A story of solving problems before they start

Amir, a computer science student, had just started his first major project — a web-based quiz system. He wrote code every day and made fast progress, but as the project grew so did its complexity. One day he changed several files to add a login feature. After testing he discovered that not only was the new feature broken, it had also broken parts of the program that had previously worked. He tried to undo the changes by hand, but could no longer remember what the original code looked like.

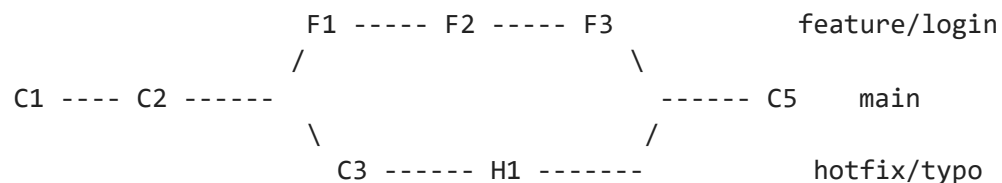
A friend introduced him to **Git**. With Git, Amir could *commit* changes as he worked, like saving checkpoints in a game; each commit is a safe, restorable state of the code. If something went wrong he could return to an earlier version — an unlimited undo button for the entire project. He then discovered **GitHub**, which stores Git projects online, so that his work was backed up in the cloud, could be shared with classmates, and could be shown publicly as a portfolio.

### 3.2 Branches and nodes: exploring without losing the way

As Amir grew more confident he wanted to try new features — a dark mode, a new grading algorithm — without endangering the working version. This is what **branches** are for.

Think of the project as a tree: the **main** branch is the trunk, the stable working version. A new branch is a new limb: a separate line of development where changes can be made and tested freely. Every commit on any branch is stored as a **node** — a snapshot in the history. If the experiment succeeds, the branch is *merged* back into the trunk; if it fails, the branch is deleted and the trunk is untouched.

The picture that results is called the **Git graph**:



### Task 1.5 — Install Git and configure your identity

Download Git from <https://git-scm.com/downloads> and install it with the default options. Then set the identity that will be stamped on every commit.

```
In [11]: # =====
# RUN IN A TERMINAL. Shown commented so the cell is safe to execute.
# =====

# git --version
#
# git config --global user.name "Your Full Name"
# git config --global user.email "your.email@university.edu"
# git config --global init.defaultBranch main
#
# git config --list          # confirm the settings were written

# You can also check from here, if git is on your PATH:
!git --version
```

```
git version 2.46.0.windows.1
```

## Task 1.6 — Create the laboratory portfolio repository

Create a repository named **ai-lab-portfolio** on GitHub. Every laboratory in this manual is committed to a directory of its own inside this single repository. It will be the artefact you deploy in Laboratory 15 and defend in Laboratory 16.

```
In [12]: # =====
# RUN IN A TERMINAL, from wherever you keep your coursework.
# =====

# mkdir ai-lab-portfolio && cd ai-lab-portfolio
# git init
#
# mkdir lab01
# # move lab01_setup.ipynb into lab01/
#
# # Never commit data, environments or secrets.
# # Create .gitignore in your editor with exactly these lines:
# #     .ipynb_checkpoints/
# #     __pycache__/
# #     .venv/
# #     env/
# #     data/
# #     *.pkl
# #     .env
#
# git add .
# git commit -m "Lab 01: environment setup and first notebook"
#
# git remote add origin https://github.com/<your-username>/ai-lab-portfolio.git
# git branch -M main
# git push -u origin main

print("Portfolio repository created. Paste your repository URL in the cell below.")
```

Portfolio repository created. Paste your repository URL in the cell below.

```
In [13]: # Record your repository URL here so the instructor can find it.
REPOSITORY_URL = "https://github.com/<your-username>/ai-lab-portfolio"

print("Submitted repository:", REPOSITORY_URL)
```

Submitted repository: https://github.com/<your-username>/ai-lab-portfolio

## The branch → commit → merge cycle

This is the cycle every later laboratory uses. Practise it now on something trivial.

```
In [14]: # =====
# RUN IN A TERMINAL, inside ai-lab-portfolio.
# =====

# git checkout -b Lab01/inspection      # create and switch to a branch
# # ... edit the notebook ...
# git add Lab01/Lab01_setup.ipynb
# git commit -m "Add describe() analysis of the Titanic dataset"
#
# git checkout main
# git merge Lab01/inspection            # bring the work back into the trunk
# git branch -d Lab01/inspection        # delete the branch once merged
# git log --oneline --graph --all       # inspect the graph you have just built

print("Paste the output of 'git log --oneline --graph --all' into the next cell.")
```

Paste the output of 'git log --oneline --graph --all' into the next cell.

## Command reference

| Command                                       | Effect                                                                           |
|-----------------------------------------------|----------------------------------------------------------------------------------|
| <code>git status</code>                       | Shows which files are modified, staged or untracked. Run it before every commit. |
| <code>git add &lt;path&gt;</code>             | Stages a change for the next commit.                                             |
| <code>git commit -m "..."</code>              | Creates a new node in the history with the staged changes.                       |
| <code>git log --oneline --graph</code>        | Draws the Git graph in the terminal.                                             |
| <code>git checkout -b &lt;name&gt;</code>     | Creates a branch and switches to it.                                             |
| <code>git merge &lt;name&gt;</code>           | Merges the named branch into the current one.                                    |
| <code>git push</code> / <code>git pull</code> | Synchronises the local history with GitHub in each direction.                    |

## 4. In-Lab Exercises

Complete all four during the session and demonstrate them to the instructor.

**Exercise 1.** Create the `ai-lab` environment, register it as a Jupyter kernel, and produce a screenshot showing the kernel selected inside a notebook.

**Exercise 2.** Load the Titanic CSV from its URL and report, **from `info()` alone**, which three columns contain missing values and how many are missing in each.

**Exercise 3.** Using `describe()`, identify the column with the largest ratio of standard deviation to mean, and explain in two sentences why that matters for a model that has not been given scaled inputs.

**Exercise 4.** Make three commits on a branch named `lab01/exercises`, merge it into `main`, and paste the output of `git log --oneline --graph --all` into your report.

```
In [15]: # --- Exercise 2: your answer -----
# Use df.info() above, then fill in this dictionary from what you read.

missing_columns = {
    # "column_name": number_missing,
}

print("Columns with missing values (your answer):", missing_columns)
print()
print("Checked against pandas:")
print(df.isnull().sum()[df.isnull().sum() > 0])
```

Columns with missing values (your answer): {}

Checked against pandas:

```
age          177
embarked      2
deck        688
embark_town   2
dtype: int64
```

```
In [16]: # --- Exercise 3: coefficient of variation -----
# std / mean tells you how spread a column is RELATIVE to its own size.
# A model without scaling treats a large-variance column as more important.

numeric = df.select_dtypes(include="number")
cv = (numeric.std() / numeric.mean()).abs().sort_values(ascending=False)

print("Coefficient of variation (std / mean), largest first:\n")
print(cv.round(3))
print()
print("Largest:", cv.index[0])
```

Coefficient of variation (std / mean), largest first:

```
parch      2.112
sibsp      2.108
fare       1.543
survived   1.268
age        0.489
pclass     0.362
dtype: float64
```

Largest: parch

**Your two-sentence answer to Exercise 3:**

*(double-click this cell and write your answer here)*

---



---

## 5. Home Assignment

Write `lab01/inspect.py` containing a function `inspect(path_or_url)` that loads a CSV and prints a single compact report: shape, dtypes, per-column missing counts and percentages, and the numeric summary.

Run it against **two different public datasets**, commit the script and both reports on a branch named `lab01/homework`, open the branch on GitHub, and merge it. Submit the URL of the merge commit.

A skeleton is provided below — complete it, then save it as `inspect.py`.

```
In [17]: def inspect(path_or_url):
    """Load a CSV and print a compact, standard first-look report.

    Parameters
    -----
    path_or_url : str
        A local file path or an http(s) URL pointing at a CSV file.
    """
    import pandas as pd

    df = pd.read_csv(path_or_url)

    print("=" * 70)
    print("SOURCE :", path_or_url)
    print("SHAPE :", df.shape[0], "rows x", df.shape[1], "columns")
    print("=" * 70)

    print("\n--- DATA TYPES ---")
    print(df.dtypes)

    print("\n--- MISSING VALUES ---")
```

```
miss = df.isnull().sum()
miss = miss[miss > 0]
if miss.empty:
    print("None.")
else:
    report = pd.DataFrame({
        "missing": miss,
        "percent": (miss / len(df) * 100).round(1),
    })
    print(report.sort_values("missing", ascending=False))

print("\n--- NUMERIC SUMMARY ---")
print(df.describe())

return df

# Test it on the Titanic dataset
_ = inspect("https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.")
```

```
=====
SOURCE : https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv
SHAPE  : 891 rows x 15 columns
=====
```

```
--- DATA TYPES ---
```

```
survived      int64
pclass        int64
sex           str
age           float64
sibsp         int64
parch         int64
fare          float64
embarked      str
class         str
who           str
adult_male    bool
deck          str
embark_town   str
alive         str
alone         bool
dtype: object
```

```
--- MISSING VALUES ---
```

```
      missing  percent
deck         688     77.2
age          177     19.9
embarked      2       0.2
embark_town   2       0.2
```

```
--- NUMERIC SUMMARY ---
```

|       | survived   | pclass     | age        | sibsp      | parch      | fare       |
|-------|------------|------------|------------|------------|------------|------------|
| count | 891.000000 | 891.000000 | 714.000000 | 891.000000 | 891.000000 | 891.000000 |
| mean  | 0.383838   | 2.308642   | 29.699118  | 0.523008   | 0.381594   | 32.204208  |
| std   | 0.486592   | 0.836071   | 14.526497  | 1.102743   | 0.806057   | 49.693429  |
| min   | 0.000000   | 1.000000   | 0.420000   | 0.000000   | 0.000000   | 0.000000   |
| 25%   | 0.000000   | 2.000000   | 20.125000  | 0.000000   | 0.000000   | 7.910400   |
| 50%   | 0.000000   | 3.000000   | 28.000000  | 0.000000   | 0.000000   | 14.454200  |
| 75%   | 1.000000   | 3.000000   | 38.000000  | 1.000000   | 0.000000   | 31.000000  |
| max   | 1.000000   | 3.000000   | 80.000000  | 8.000000   | 6.000000   | 512.329200 |

## 6. Deliverables

Commit the following to `lab01/` in your `ai-lab-portfolio` repository:

| File                                 | Contents                                                               |
|--------------------------------------|------------------------------------------------------------------------|
| <code>lab01/lab01_setup.ipynb</code> | This notebook, executed, with all outputs visible                      |
| <code>lab01/inspect.py</code>        | The completed <code>inspect()</code> function from the home assignment |

| File             | Contents                                                               |
|------------------|------------------------------------------------------------------------|
| requirements.txt | pip freeze > requirements.txt from the ai-lab environment              |
| .gitignore       | Excluding data, environments and secrets                               |
| lab01/report.md  | Installation evidence, the three inspection outputs, and the Git graph |

## Assessment Rubric

| Criterion                        | Weight | Excellent performance                                     |
|----------------------------------|--------|-----------------------------------------------------------|
| Correctness of implementation    | 35 %   | Code runs unmodified and produces the specified artefacts |
| Methodological soundness         | 25 %   | Inspections performed before any modification of the data |
| Analysis and interpretation      | 20 %   | Results are explained, not merely printed                 |
| Code quality and reproducibility | 10 %   | Modular, documented, pinned requirements.txt              |
| Report and demonstration         | 10 %   | Clear written report and a working live demonstration     |

**Next laboratory:** Lab 02 — Web Scraping, Feature Engineering and Exploratory Data Analysis.